

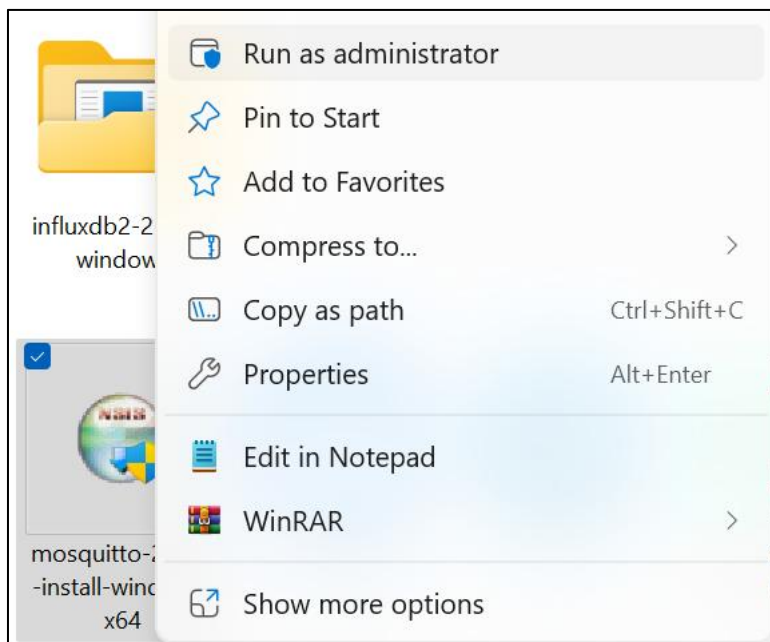
Installation of MQTT Broker in Windows

Steps to be followed

Go to Mosquitto Download WebPage - <https://mosquitto.org/download/>

Select <https://mosquitto.org/files/binary/win64/mosquitto-2.0.22-install-windows-x64.exe>

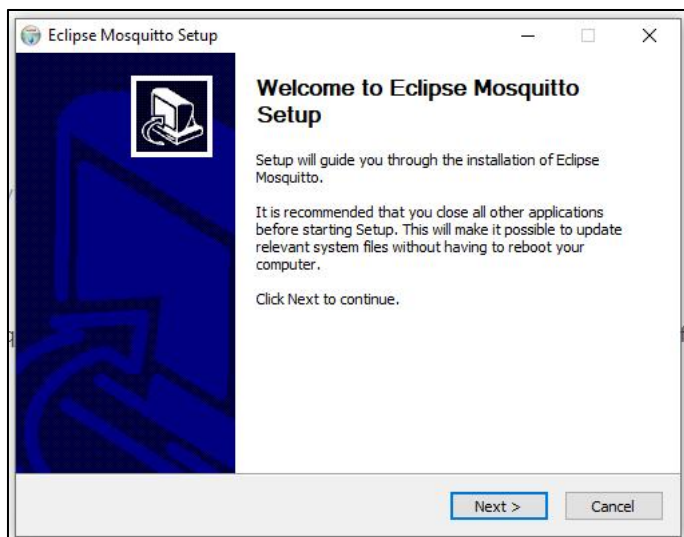
Select -Run as administrator



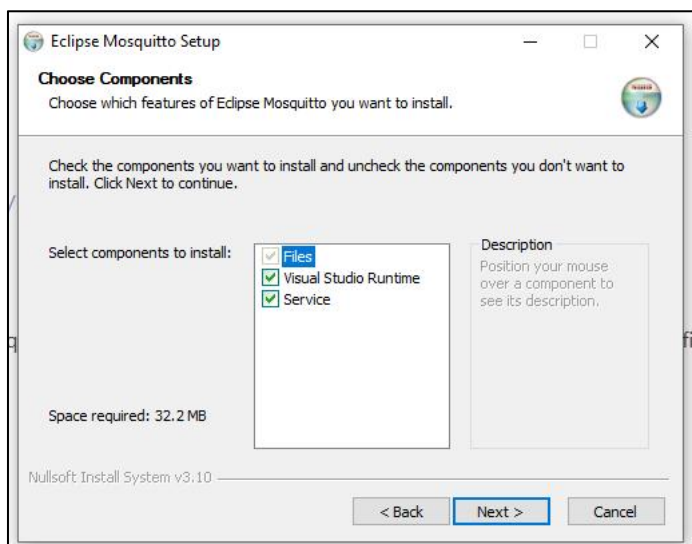
Select - Run Anyway



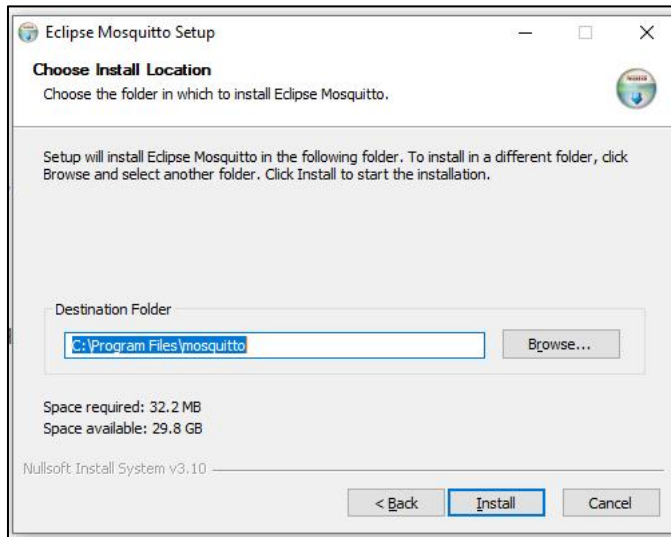
Select - Next



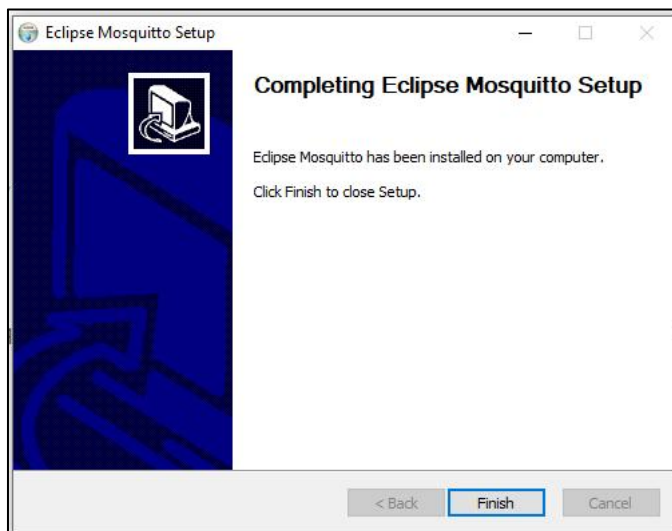
Select - Next



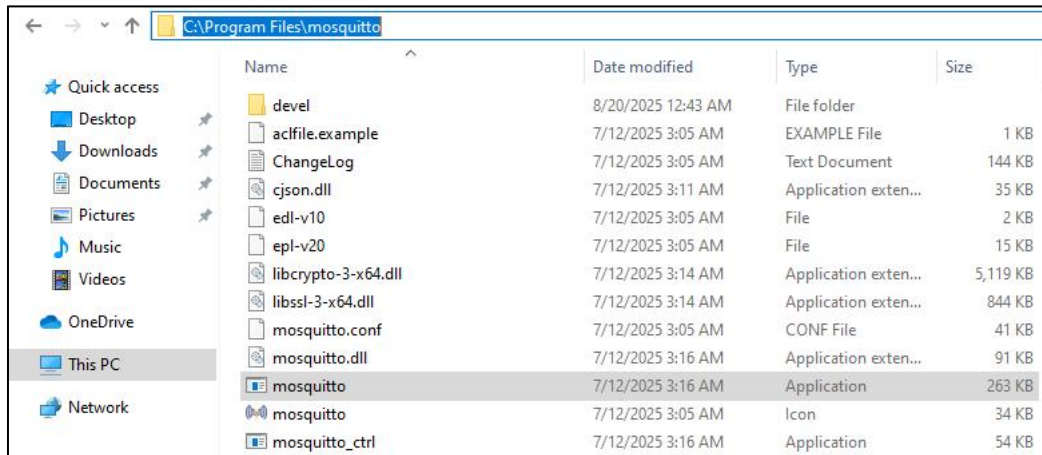
Select-Install (if in case there is a need to change in destination folder then change it otherwise select the default)



Select -Finish



Go to File Location - C:\Program Files\mosquitto we can see the application is available



To check the application is installed correctly launch CMD(Command Prompt) run the command `cd C:\Program Files\mosquitto`

Type `mosquitto -h` and enter, now we can observe the details of which version of mosquitto installed

```
Microsoft Windows [Version 10.0.19045.3803]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Sanjay>cd C:\Program Files\mosquitto

C:\Program Files\mosquitto>mosquitto -h
mosquitto version 2.0.22

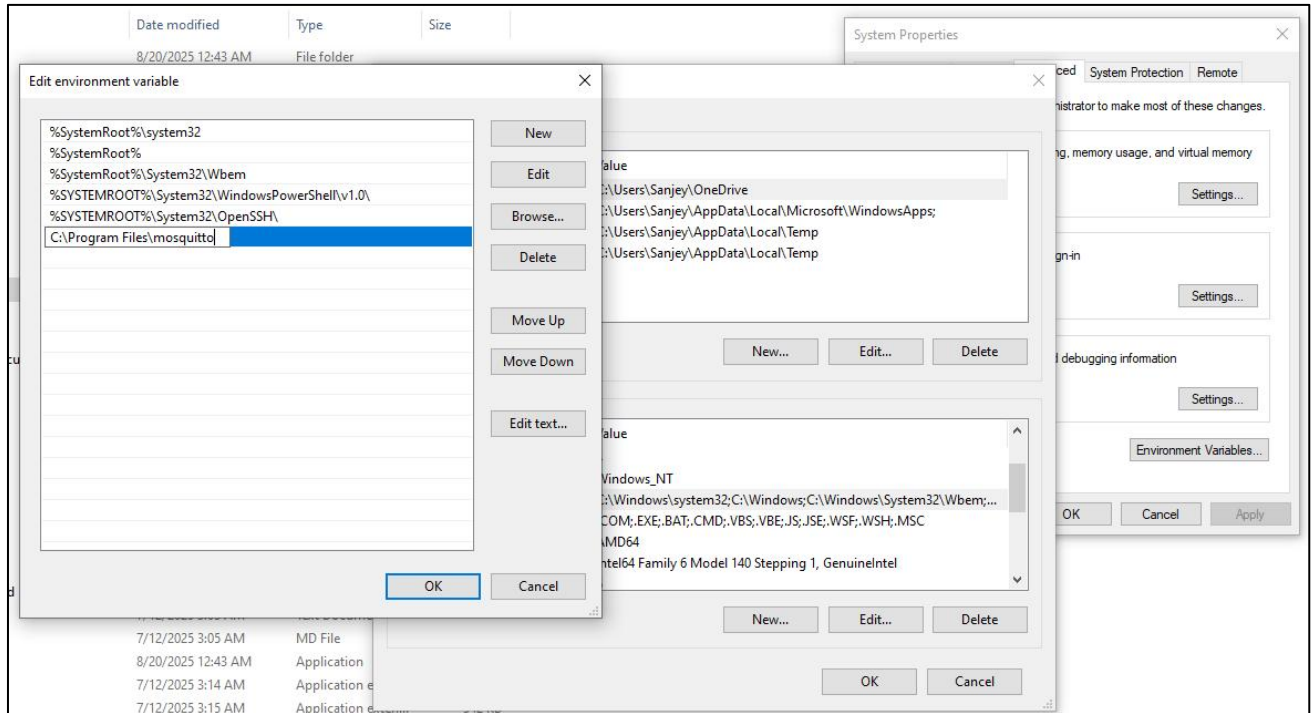
mosquitto is an MQTT v5.0/v3.1.1/v3.1 broker.

Usage: mosquitto [-c config_file] [-d] [-h] [-p port]

-c : specify the broker config file.
-d : put the broker into the background after starting.
-h : display this help.
-p : start the broker listening on the specified port.
    Not recommended in conjunction with the -c option.
-v : verbose mode - enable all logging types. This overrides
    any logging options given in the config file.

See https://mosquitto.org/ for more information.
```

We need to make mosquitto client globally so enter environment variables, inside the system environmental variables search for path select it and edit it. Click on new and paste the mosquitto client path, then click OK.



Open CMD and run `mosquitto -h` now the mosquitto client is globally available

```
Command Prompt
Microsoft Windows [Version 10.0.19045.3803]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Sanjey>mosquitto -h
mosquitto version 2.0.22

mosquitto is an MQTT v5.0/v3.1.1/v3.1 broker.

Usage: mosquitto [-c config_file] [-d] [-h] [-p port]

-c : specify the broker config file.
-d : put the broker into the background after starting.
-h : display this help.
-p : start the broker listening on the specified port.
    Not recommended in conjunction with the -c option.
-v : verbose mode - enable all logging types. This overrides
    any logging options given in the config file.

See https://mosquitto.org/ for more information.
```

Installation for mosquitto client is finished !

Detailed Video based explanation - [video](#)

Two CMD Tabs – MQTT Publish/Subscribe Demo

To show how Mosquitto handles message exchange using the **MQTT protocol**, which is essential for IoT devices like ESP32 or Arduino.

Open two cmd tabs and run the following

in the first cmd tab run - `mosquitto_pub -h localhost -t myMessage -m "HelloWorld!" -q 1`

In the second cmd tab run - `mosquitto_sub -h localhost -t myMessage -q 1`

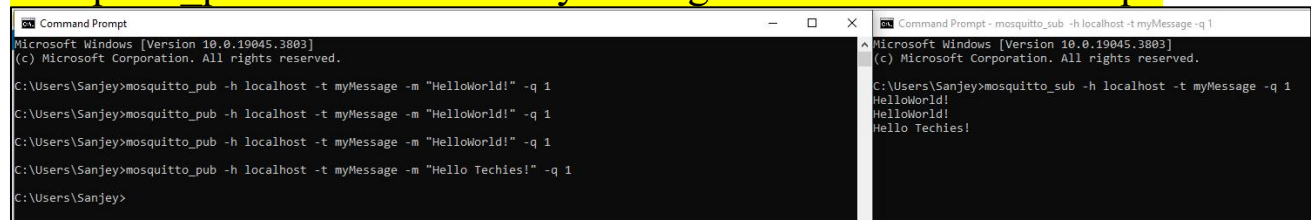
Now both the cmd tabs will act as publisher and subscriber

run these commands in the first cmd tab we can observe the msg sent are received in the second cmd tab.

`mosquitto_pub -h localhost -t myMessage -m "HelloWorld!" -q 1`

`mosquitto_pub -h localhost -t myMessage -m "HelloWorld!" -q 1`

`mosquitto_pub -h localhost -t myMessage -m "Hello Techies!" -q 1`



```
Command Prompt
Microsoft Windows [Version 10.0.19045.3803]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Sanjay>mosquitto_pub -h localhost -t myMessage -m "HelloWorld!" -q 1
C:\Users\Sanjay>mosquitto_pub -h localhost -t myMessage -m "HelloWorld!" -q 1
C:\Users\Sanjay>mosquitto_pub -h localhost -t myMessage -m "HelloWorld!" -q 1
C:\Users\Sanjay>mosquitto_pub -h localhost -t myMessage -m "Hello Techies!" -q 1
C:\Users\Sanjay>

Command Prompt - mosquitto_sub -h localhost -t myMessage -q 1
Microsoft Windows [Version 10.0.19045.3803]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Sanjay>mosquitto_sub -h localhost -t myMessage -q 1
HelloWorld!
HelloWorld!
HelloWorld!
Hello Techies!
```

Installation of Node-Red

Install Node.js -[windows installer\(.msi\)](#)

Or get a prebuilt Node.js® for Windows running a x64 architecture.

 [Windows Installer \(.msi\)](#)

 [Standalone Binary \(.zip\)](#)

Read the [changelog](#) or [blog post](#) for this version.

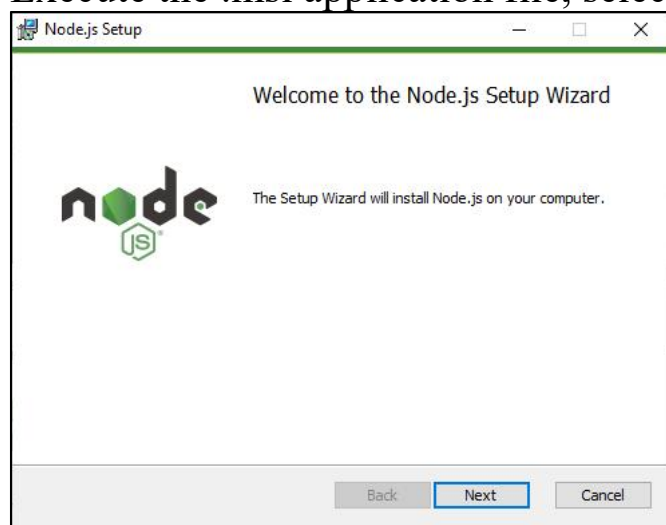
Learn more about [Node.js releases](#), including the release schedule and LTS status.

Learn how to [verify](#) signed SHASUMS.

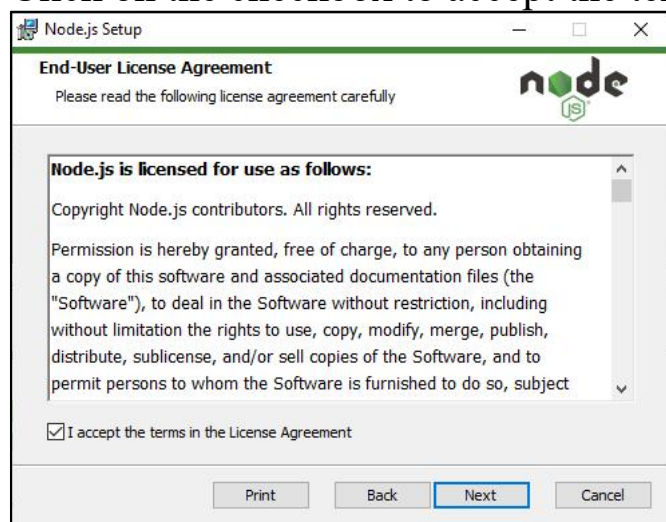
Looking for Node.js source? Download a signed [Node.js source](#) tarball.

Check out our [nightly](#) binaries or all [previous releases](#) or the [unofficial](#) binaries for other platforms.

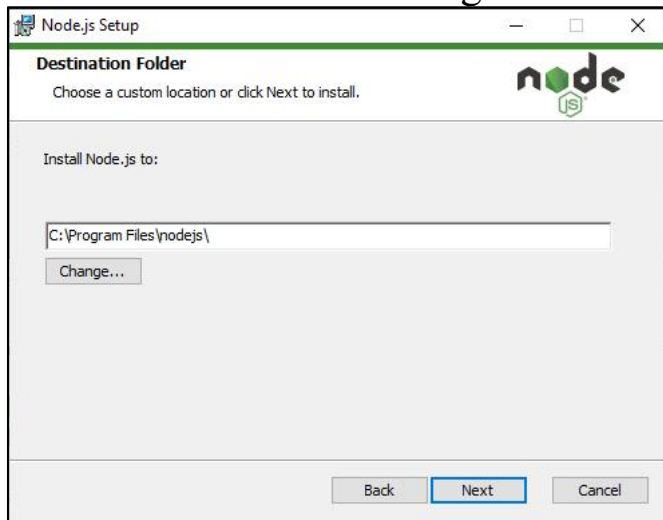
Execute the .msi application file, select next



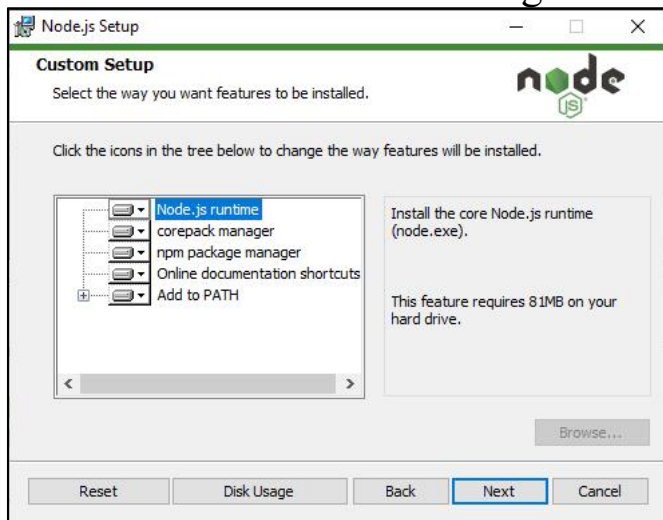
Click on the checkbox to accept the terms and conditions. Select next



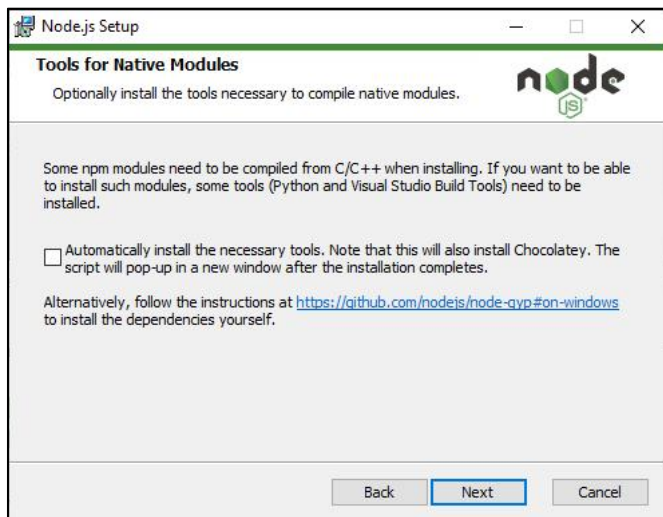
Install node.js to the default destination folder or else if you want to change the destination folder change it. Select next



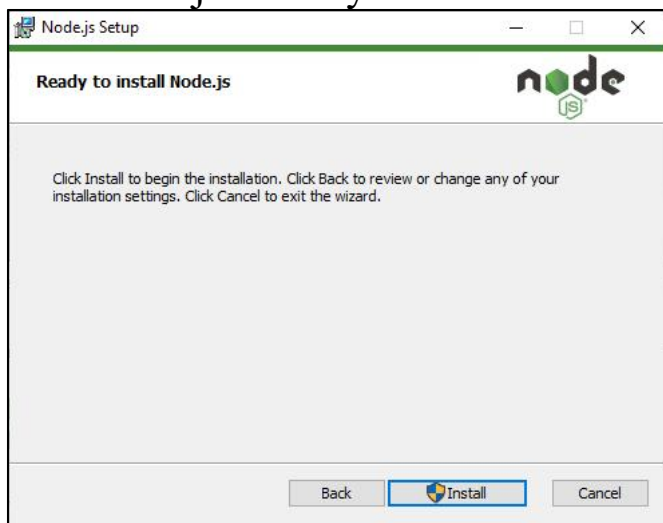
Select next as there is no change in here.



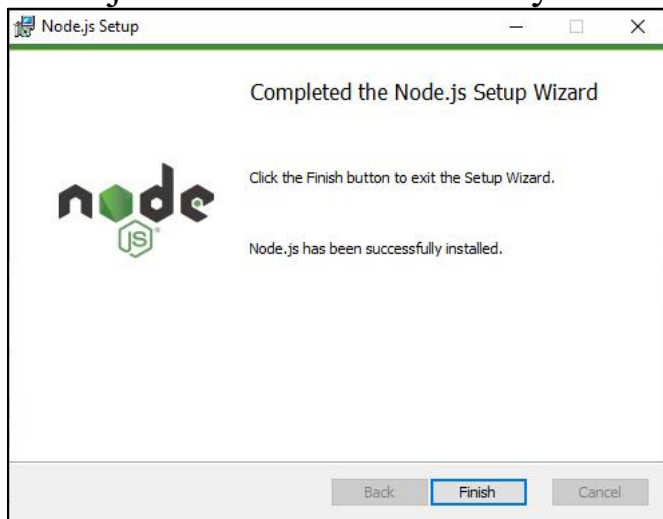
Select next. Don't click on the checkbox.



Now node.js is ready to install so select install option.



Node.js is installed successfully. Click finish to close.



Now It is required to install node-red for windows so go to the site mentioned - <https://nodered.org/docs/getting-started/windows>

In the web-page mentioned go to Step-2 we can see a command. We need to execute it so copy and paste the command in command prompt.

2. Install Node-RED

Installing Node-RED as a global module adds the command `node-red` to your system path. Execute the following at the command prompt:

```
npm install -g --unsafe-perm node-red
```

Once completed the installation close the Command Prompt. Now open a new Command Prompt then type node-red and then enter. We can observe the welcome note for us.

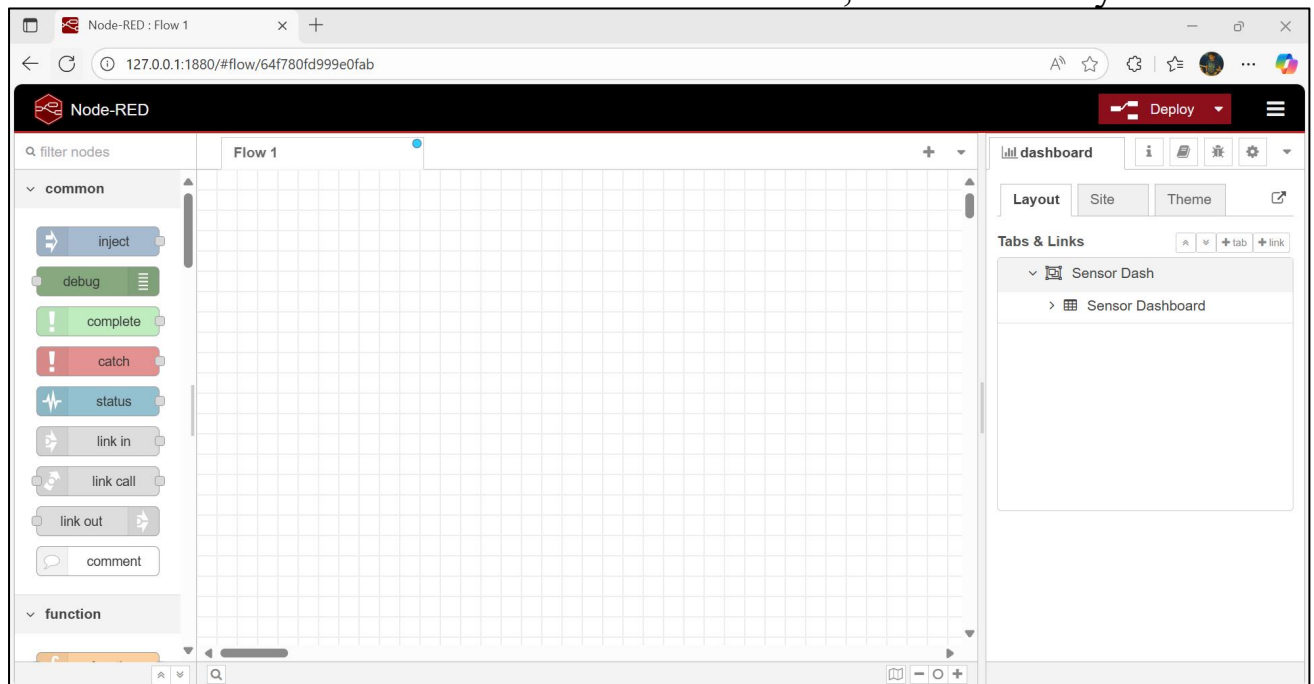
```
node-red
Microsoft Windows [Version 10.0.26100.4946]
(c) Microsoft Corporation. All rights reserved.

C:\Users\super>node-red
20 Aug 11:06:33 - [info]

Welcome to Node-RED
=====

20 Aug 11:06:33 - [info] Node-RED version: v4.1.0
20 Aug 11:06:33 - [info] Node.js version: v22.18.0
20 Aug 11:06:33 - [info] Windows_NT 10.0.26100 x64 LE
20 Aug 11:06:33 - [info] Loading palette nodes
20 Aug 11:06:34 - [info] Dashboard version 3.6.6 started at /ui
20 Aug 11:06:34 - [info] Settings file : C:\Users\super\.node-red\settings.js
20 Aug 11:06:34 - [info] Context store : 'default' [module=memory]
20 Aug 11:06:34 - [info] User directory : \Users\super\.node-red
20 Aug 11:06:34 - [warn] Projects disabled : editorTheme.projects.enabled=false
20 Aug 11:06:34 - [info] Flows file : \Users\super\.node-red\flows.json
20 Aug 11:06:34 - [info] Server now running at http://127.0.0.1:1880/
20 Aug 11:06:34 - [warn]
```

We can observe an web-page address copy that and paste it in your default browser. We can observe the node-red dashboard, now it is ready to work.



Installation is completed !

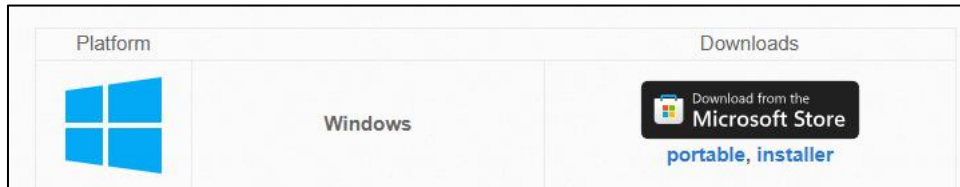
For Video Demonstration - [Video](#)

Installation of MQTT Explorer

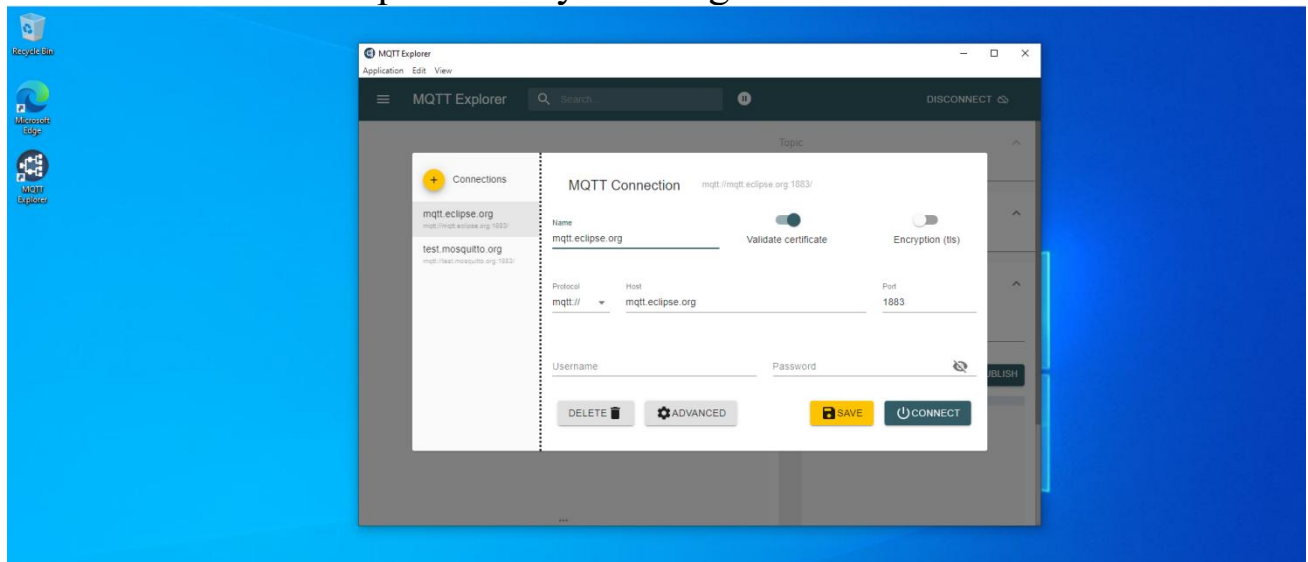
Steps to be followed

Visit - <https://mqtt-explorer.com/>

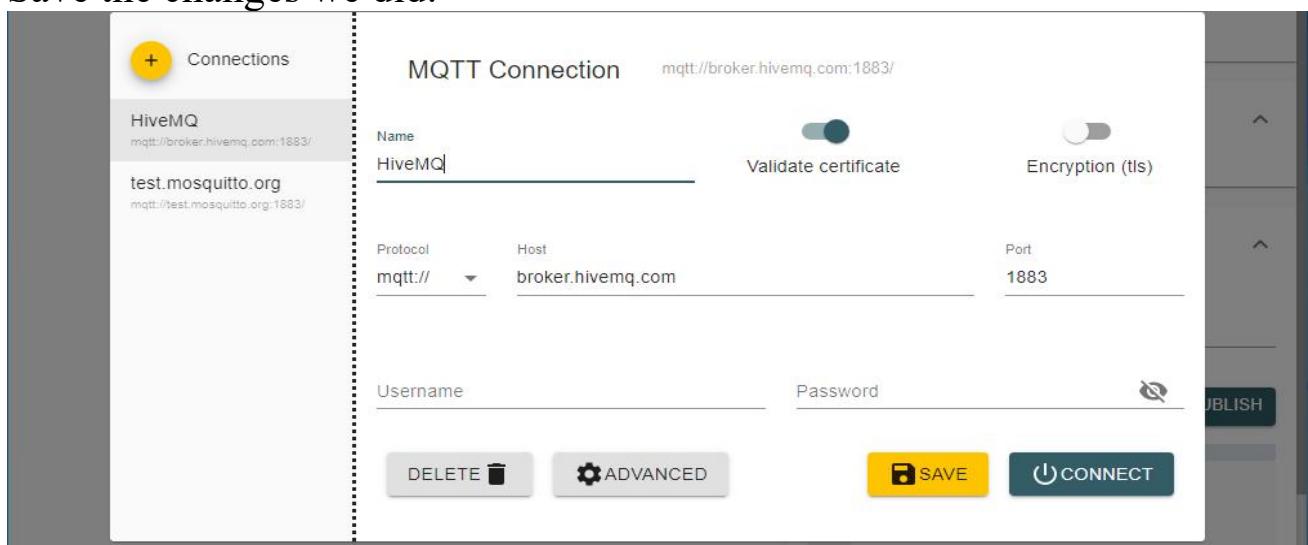
For Windows Click [installer](#) , now the file will be automatically downloaded.



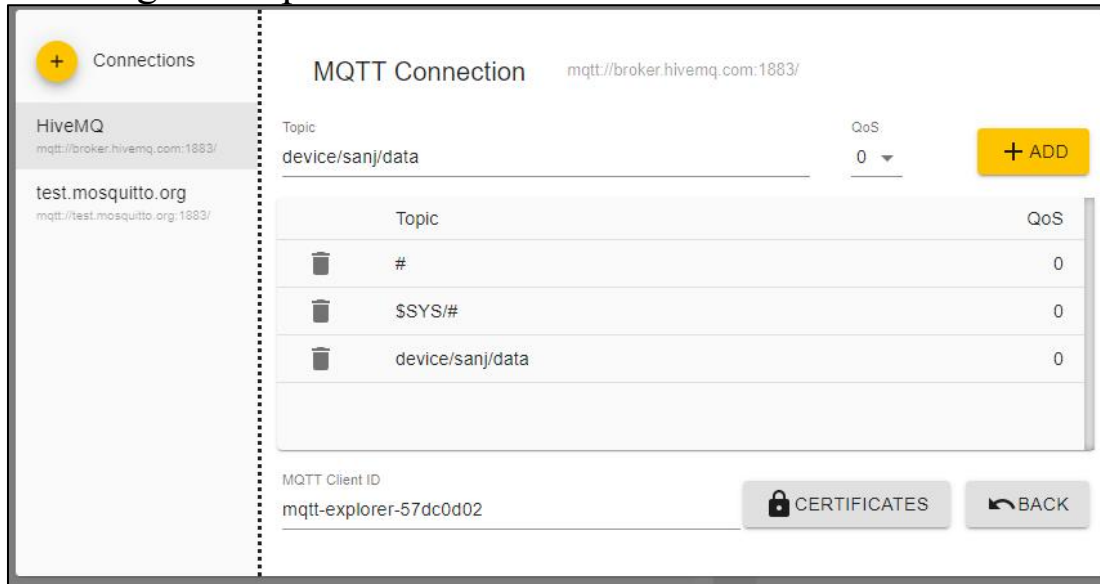
Click and execute the .exe file. Now we observe that a shortcut is available at the home screen. Open it and you will get the interface.



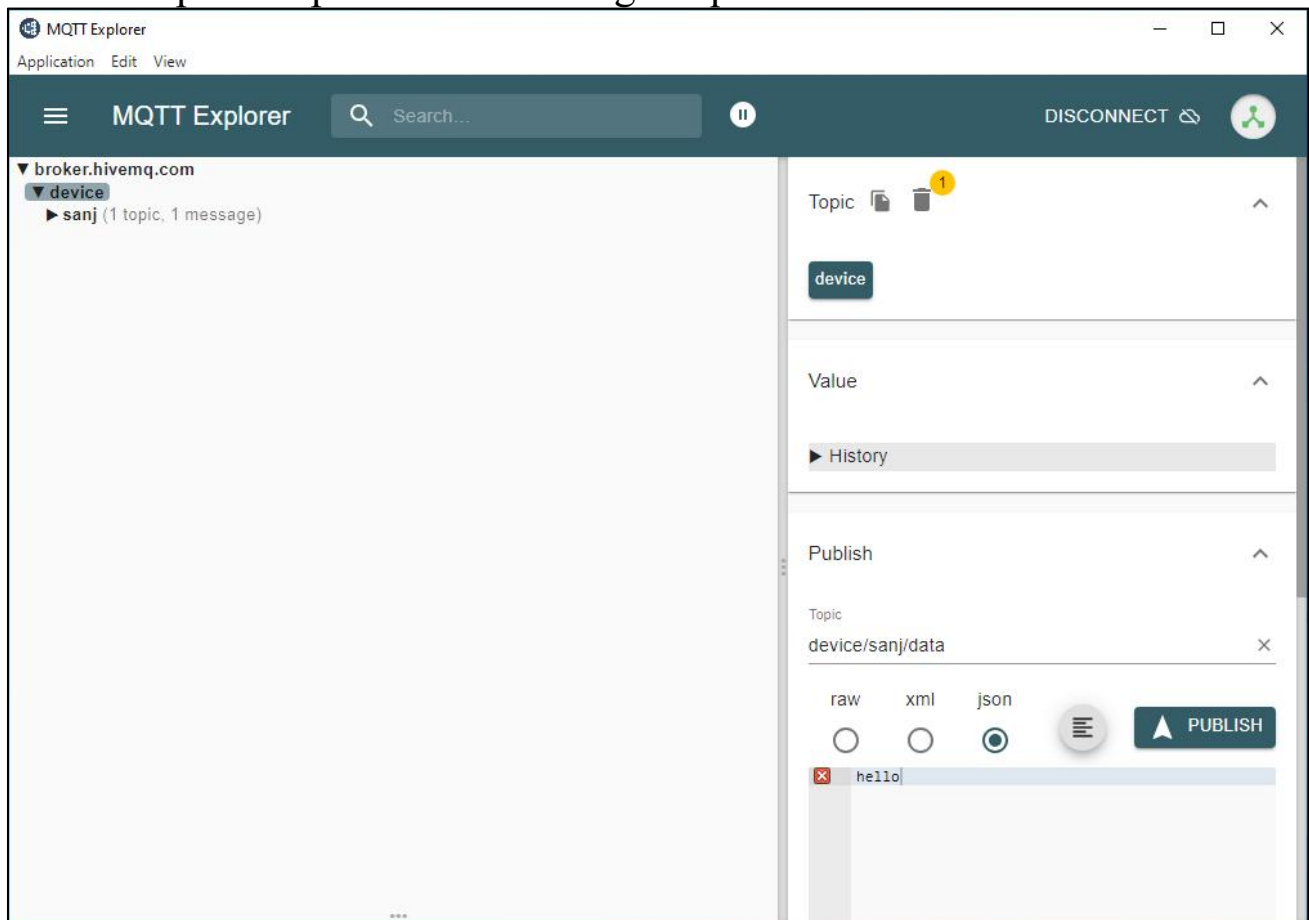
Edit name and host such as **Name - HiveMQ, Host - broker.hivemq.com**
Save the changes we did.



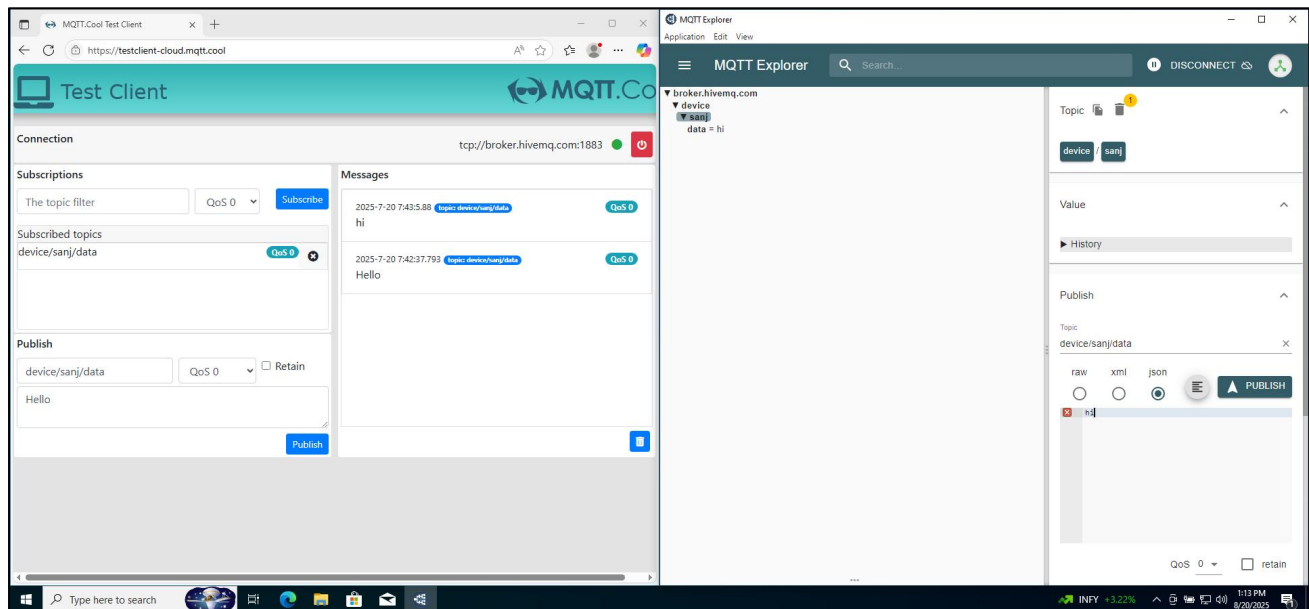
Now enter Advanced option, we are going to subscribe our own topic so enter the topic to subscribe (example - device/sanj/data). Select Add , now we can see the topic entered is added for subscription. Return back by selecting back option.



Select Connect. Once connected we get the interface to get subscribe the entered topic and publish the message required.



Now go to [MQTT Test Client](https://testclient-cloud.mqtt.cool) website. In the dropdown box menu select- **tcp://broker.hivemq.com:1883**. Select Connect, once it gets connected enter the topic in Subscription box and in Publish there is a box to enter the topic it will be the destination for the messages.



Installation of MQTT Explorer is completed !

For Video based Demo(test.mosquitto.org) - [Video](#)

For Video based Demo(broker.hivemq.com) - [Video](#)

[IoT TIG Stack with MQTT using Docker](#)

Install [Docker](#) and [Docker Compose](#) for Windows

Select Docker Desktop for Windows - x86

[Home](#) / [Manuals](#) / [Docker Desktop](#) / [Setup](#) / [Install](#) / [Windows](#)

Install Docker Desktop on Windows

Page options ▾

Docker Desktop terms

Commercial use of Docker Desktop in larger enterprises (more than 250 employees OR more than \$10 million USD in annual revenue) requires a [paid subscription](#).

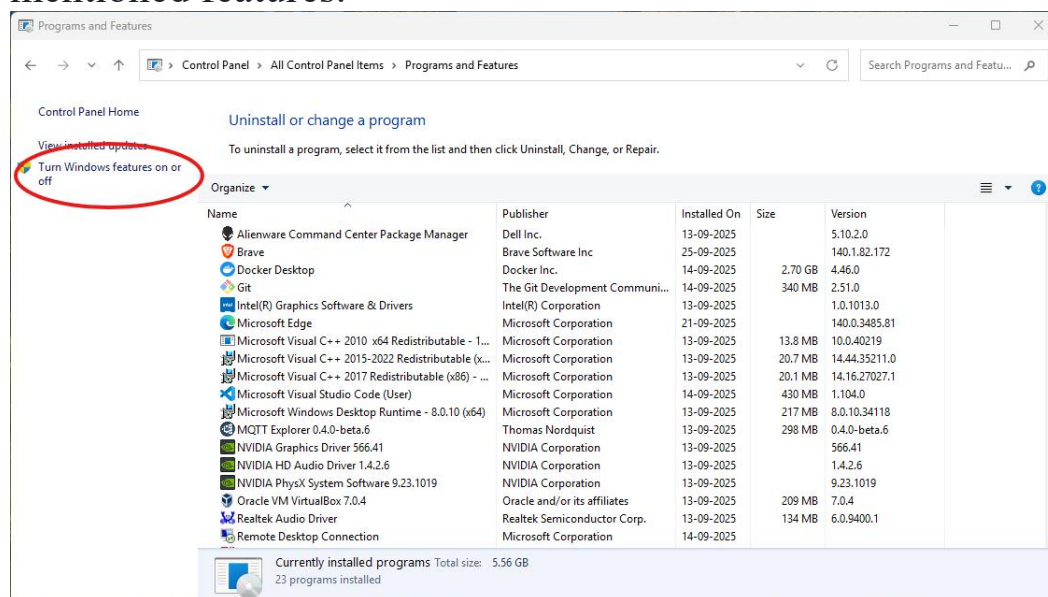
This page provides download links, system requirements, and step-by-step installation instructions for Docker Desktop on Windows.

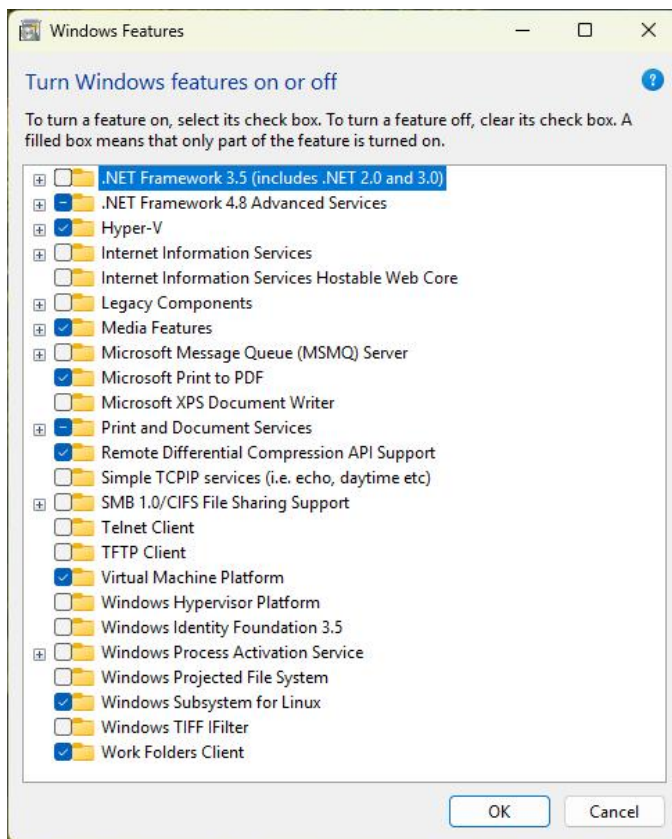
[Docker Desktop for Windows - x86_64](#)

[Docker Desktop for Windows - x86_64 on the Microsoft Store](#)

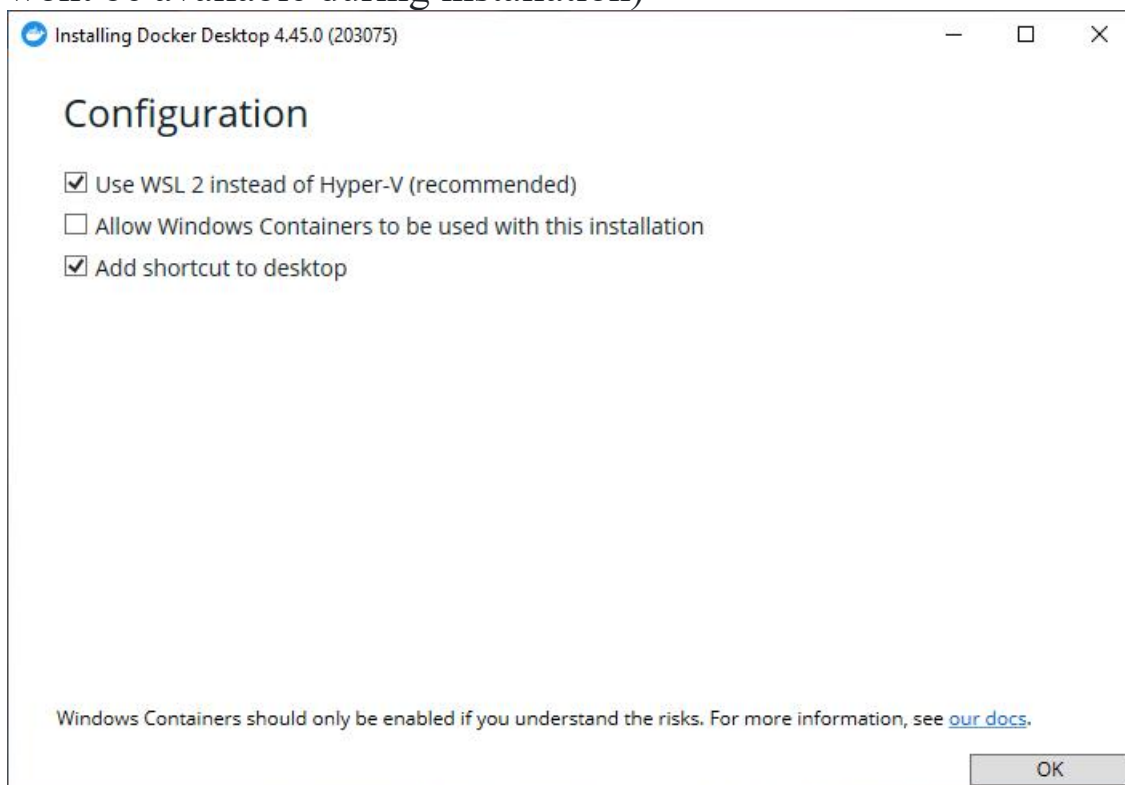
[Docker Desktop for Windows - Arm \(Early Access\)](#)

Before proceeding with the installation, ensure that the following Windows features are enabled: Hyper-V, Virtual Machine Platform, and Windows Subsystem for Linux. If these are not enabled by default, open Control Panel, select 'Turn Windows features on or off,' and check the boxes for the mentioned features.

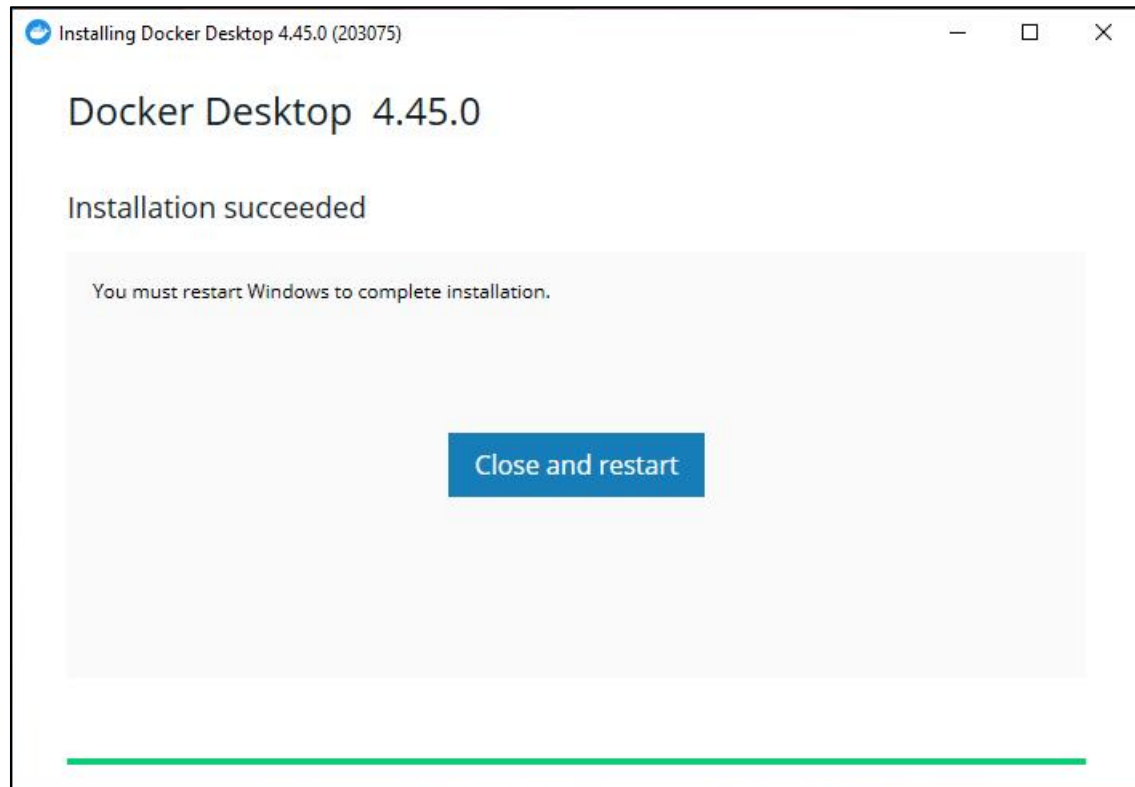




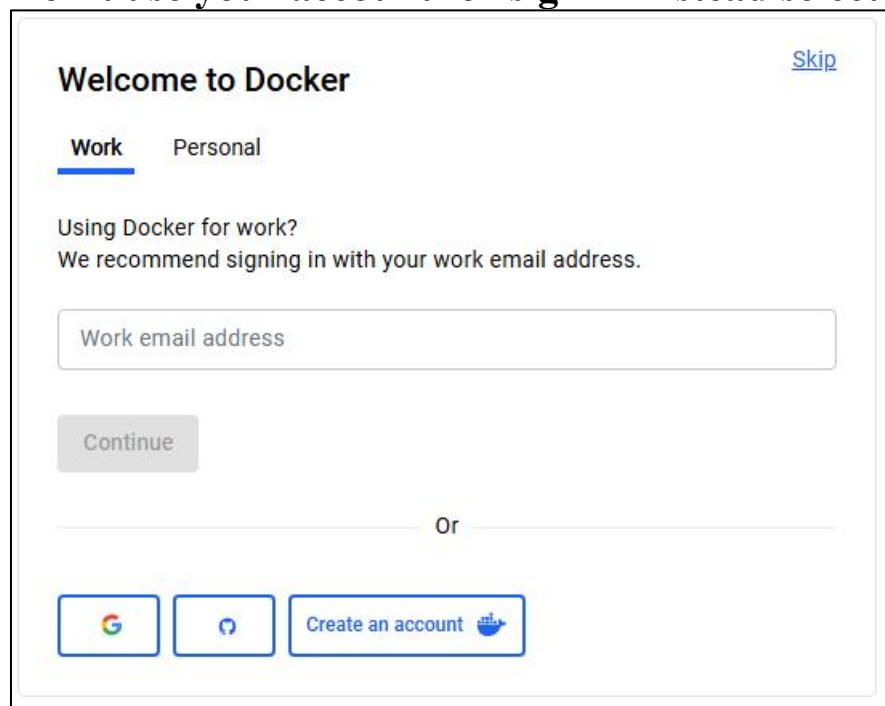
Select OK and proceed (below screenshot provides other options which wont be available during installation)



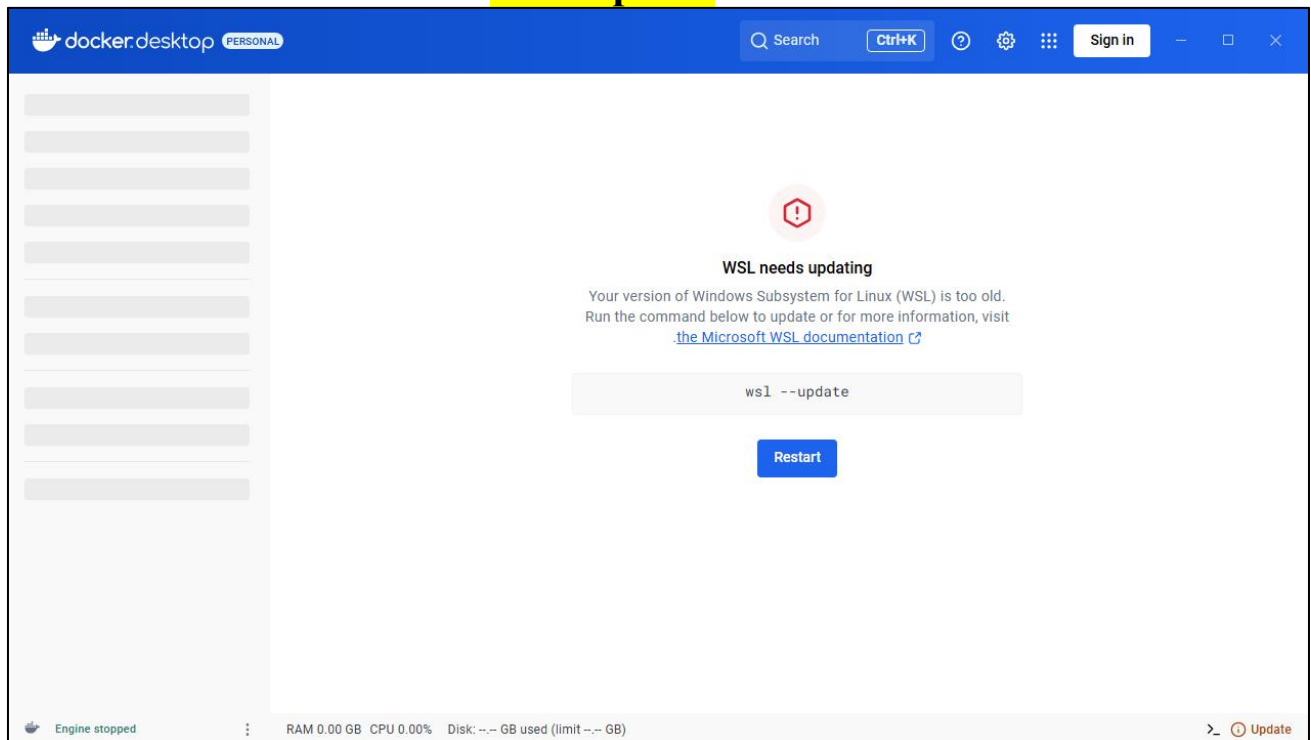
Select Close and Restart



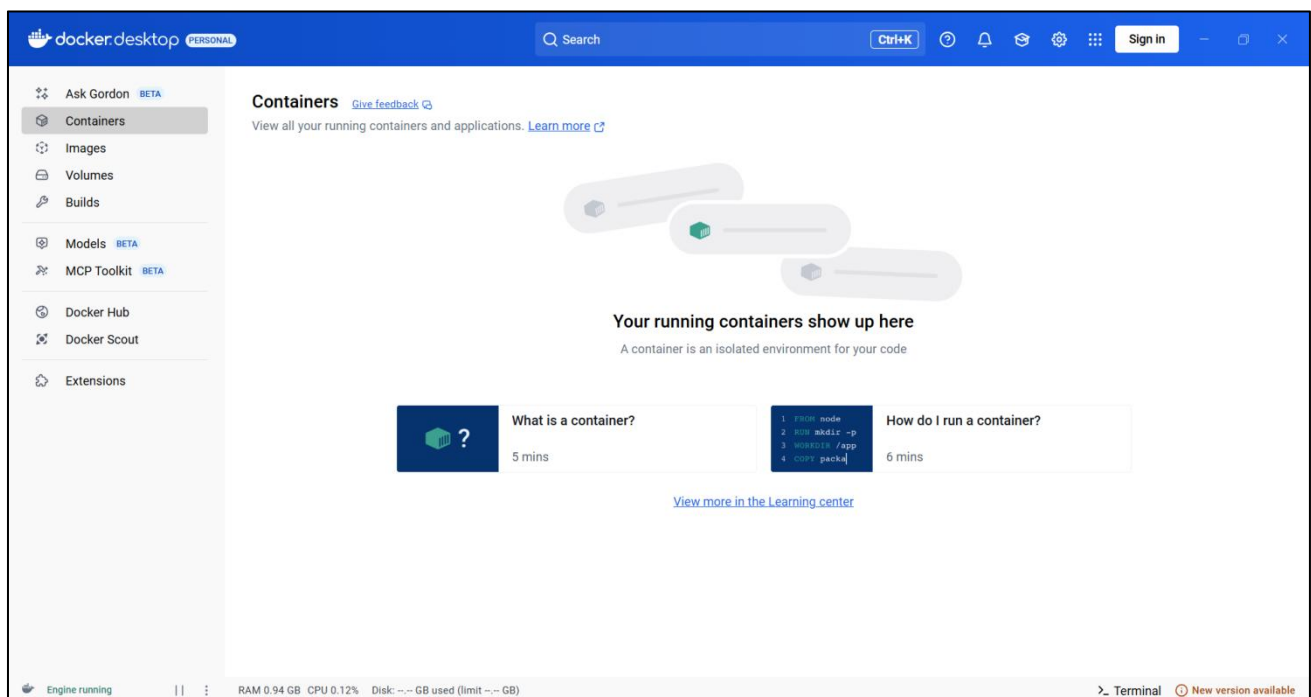
After restarting it accept **Docker Subscription Service Agreement**.
Don't use your account for sign in instead select skip and proceed.



If you have this error go to Command Prompt(Run as Administrator) and execute the command **wsl --update** .And then click restart.



After restarting Docker Engine you will get the screen without any containers.



Now we have to clone the repository [Github](https://github.com/rishikreddycheruku/iot_tig_stack_with_mqtt_using_docker.git).

so enter the command in the Command Prompt - `git clone`

`https://github.com/rishikreddycheruku/iot_tig_stack_with_mqtt_using_docker.git`

```
Command Prompt
Microsoft Windows [Version 10.0.19045.6216]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Sanjey>git clone https://github.com/rishikreddycheruku/iot_tig_stack_with_mqtt_using_docker.git
Cloning into 'iot_tig_stack_with_mqtt_using_docker'...
remote: Enumerating objects: 14, done.
remote: Counting objects: 100% (14/14), done.
remote: Compressing objects: 100% (13/13), done.
remote: Total 14 (delta 3), reused 5 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (14/14), 5.22 KiB | 534.00 KiB/s, done.
Resolving deltas: 100% (3/3), done.

C:\Users\Sanjey>
```

Execute the command in the same Command Prompt - `cd`

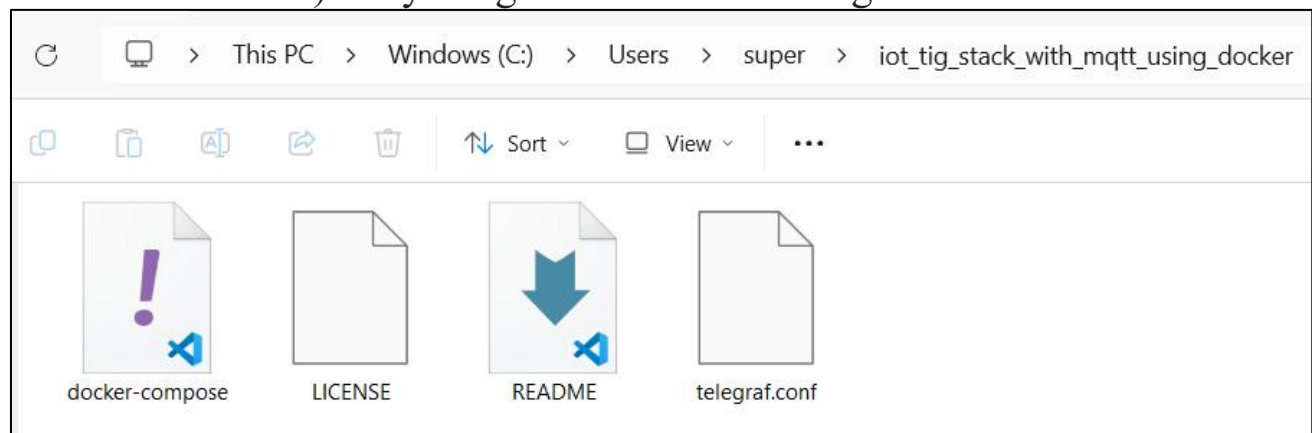
`iot_tig_stack_with_mqtt_using_docker`

```
Command Prompt
Microsoft Windows [Version 10.0.19045.6216]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Sanjey>git clone https://github.com/rishikreddycheruku/iot_tig_stack_with_mqtt_using_docker.git
Cloning into 'iot_tig_stack_with_mqtt_using_docker'...
remote: Enumerating objects: 14, done.
remote: Counting objects: 100% (14/14), done.
remote: Compressing objects: 100% (13/13), done.
remote: Total 14 (delta 3), reused 5 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (14/14), 5.22 KiB | 534.00 KiB/s, done.
Resolving deltas: 100% (3/3), done.

C:\Users\Sanjey>cd iot_tig_stack_with_mqtt_using_docker
C:\Users\Sanjey\iot_tig_stack_with_mqtt_using_docker>
```

Delete `mosquitto.conf` file as we are using `HiveMQ(broker.hivemq.com)` which is a public client whereas `test.mosquitto.com` (during testing it may occur server error). Only `telegraf.conf` file is enough



Use the code provided below for our TIG Stack as it is modified HiveMQ

telegraf.conf (code lines)

```
[agent]
  interval = "10s"
  round_interval = true

[[outputs.influxdb_v2]]
  urls = ["http://influxdb:8086"]
  token = "mysecrettoken123"
  organization = "my-org"
  bucket = "iotdb"

[[inputs.mqtt_consumer]]
  servers = ["tcp://broker.hivemq.com:1883"]
  topics = ["iot/sensor"]
  data_format = "json"
  name_override = "esp32_data"
```

docker-compose.yml (code lines)

```
version: "3.8"

services:
  influxdb:
    image: influxdb:2.7
    container_name: influxdb
    ports:
      - "8086:8086"
    volumes:
      - influxdb2_data:/var/lib/influxdb2
    environment:
      - DOCKER_INFLUXDB_INIT_MODE=setup
      - DOCKER_INFLUXDB_INIT_USERNAME=admin
      - DOCKER_INFLUXDB_INIT_PASSWORD=admin1234
      - DOCKER_INFLUXDB_INIT_ORG=my-org
      - DOCKER_INFLUXDB_INIT_BUCKET=iotdb
      - DOCKER_INFLUXDB_INIT_ADMIN_TOKEN=mysecrettoken123
```

telegraf:

image: telegraf:1.26

container_name: telegraf

depends_on:

- influxdb

volumes:

- ./telegraf.conf:/etc/telegraf/telegraf.conf:ro

grafana:

image: grafana/grafana

container_name: grafana

ports:

- "3000:3000"

environment:

- GF_SECURITY_ADMIN_USER=admin
- GF_SECURITY_ADMIN_PASSWORD=admin

volumes:

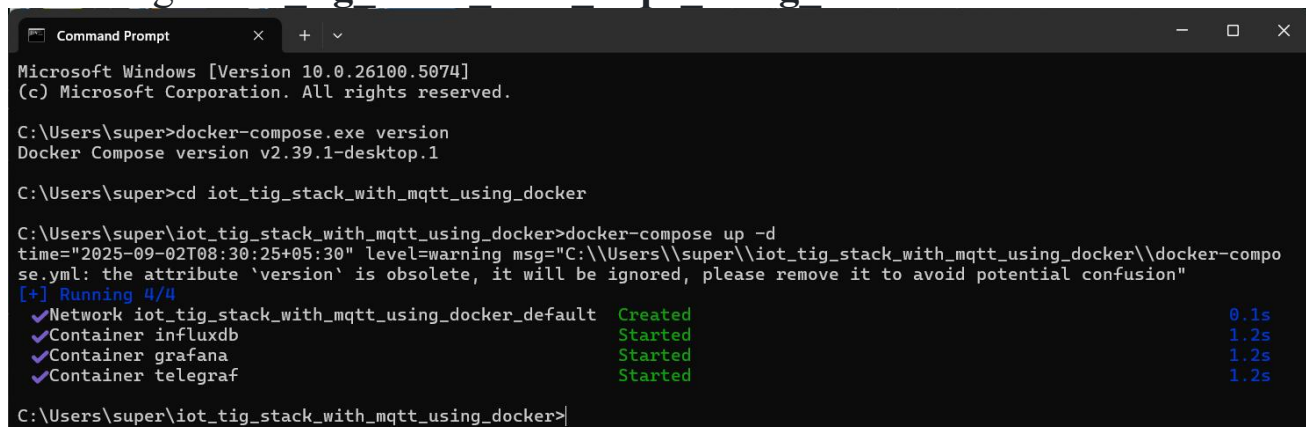
- grafana_data:/var/lib/grafana

volumes:

influxdb2_data:

grafana_data:

Run Docker Compose by executing the command in the Command Prompt - **docker-compose up -d**. Make sure that you execute the command after executing **cd iot_tig_stack_with_mqtt_using_docker**



```
Microsoft Windows [Version 10.0.26100.5074]
(c) Microsoft Corporation. All rights reserved.

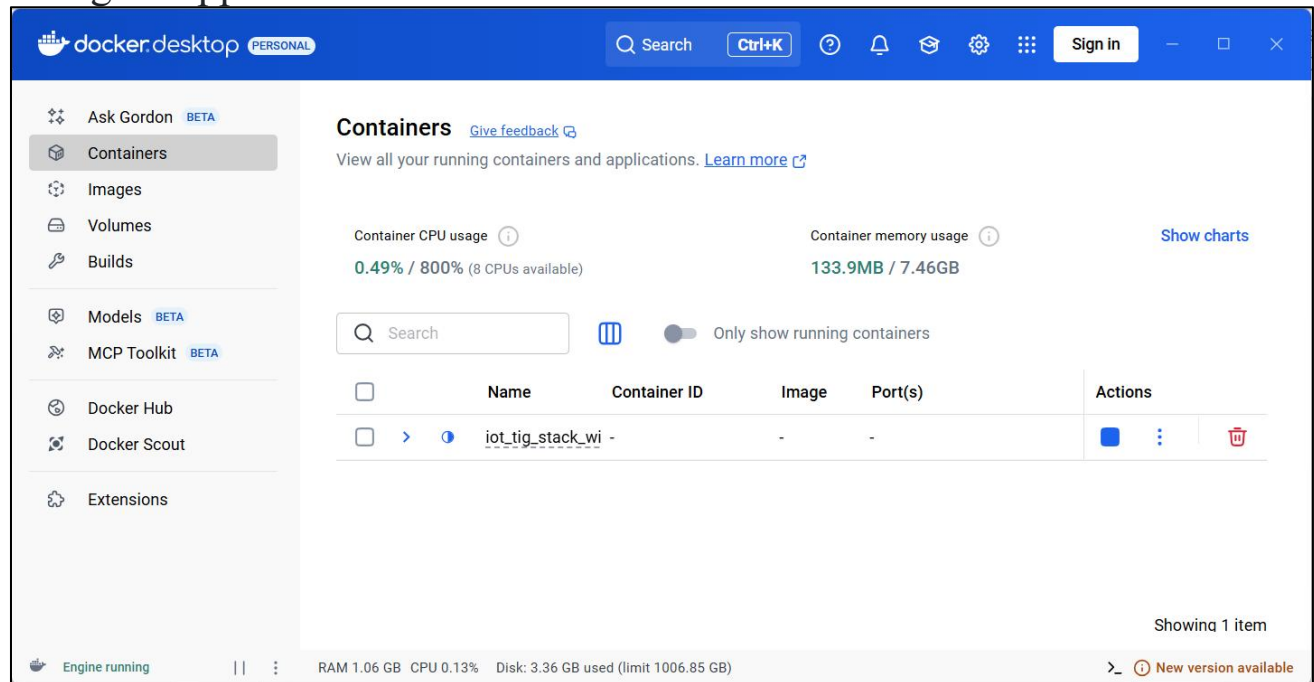
C:\Users\super>docker-compose.exe version
Docker Compose version v2.39.1-desktop.1

C:\Users\super>cd iot_tig_stack_with_mqtt_using_docker

C:\Users\super\iot_tig_stack_with_mqtt_using_docker>docker-compose up -d
time="2025-09-02T08:30:25+05:30" level=warning msg="C:\\Users\\super\\iot_tig_stack_with_mqtt_using_docker\\docker-compo
se.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Running 4/4
✔ Network iot_tig_stack_with_mqtt_using_docker_default Created 0.1s
✔ Container influxdb Started 1.2s
✔ Container grafana Started 1.2s
✔ Container telegraf Started 1.2s

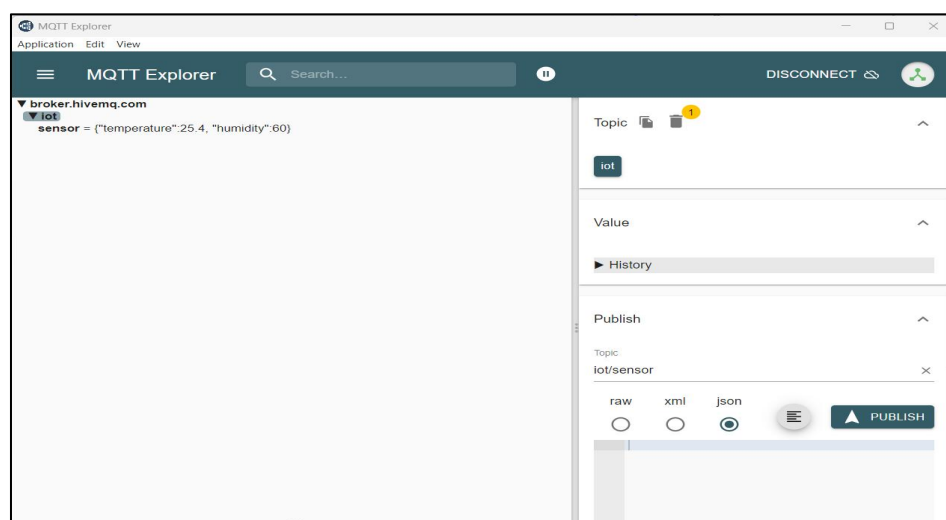
C:\Users\super\iot_tig_stack_with_mqtt_using_docker>
```

After executing command look at the Docker Engine , you can see there are changes happened



We can now publish the data to mqtt via HiveMQ, so run the command in the Command Prompt - "C:\Program Files\mosquitto\mosquitto_pub.exe" -h broker.hivemq.com -p 1883 -t iot/sensor -m '{"temperature":25.4, "humidity":60}', And the msg sent can be seen in the MQTT Explorer by subscribing the topic **iot/sensor** .

```
C:\Users\super\iot_tig_stack_with_mqtt_using_docker>"C:\Program Files\mosquitto\mosquitto_pub.exe" -h broker.hivemq.com -p 1883 -t iot/sensor -m '{"temperature":25.4, "humidity":60}'
C:\Users\super\iot_tig_stack_with_mqtt_using_docker>
```



Now we have to create Grafana Dashboard, so go to Docker Engine expand the container we can observe the ports for Grafana and InfluxDB. Click on <http://localhost:3000/> in your device and it redirects to Grafana Dashboard.

Containers [Give feedback](#)
View all your running containers and applications. [Learn more](#)

Container CPU usage
1.63% / 800% (8 CPUs available)

Container memory usage
139.82MB / 7.46GB

Show charts

☐ Only show running containers

☐	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☐	iot_tig_stack_with_mqtt_us -		-	-	3.64%	8 minutes ago	☐ ☐ ☐
☐	grafana	4d9d7bc29c23	grafana/grafana	3000:3000	3.58%	14 minutes ago	☐ ☐ ☐
☐	influxdb	0e8a9e55d770	influxdb:2.7	8086:8086	0.06%	14 minutes ago	☐ ☐ ☐
☐	telegraf	ad4d4f2c31ba	telegraf:1.26		0%	8 minutes ago	☐ ☐ ☐

Home - Dashboards - Grafana

localhost:3000/?orgId=1&from=now-6h&to=now&timezone=browser

Grafana

Home > Dashboards > Home

Q Search...

ctrl+k

Home

Bookmarks

Starred

Dashboards

Playlists

Snapshots

Library panels

Shared dashboards

Explore

Drilldown

Alerting

Connections

Add new connection

Data sources

Administration

Welcome to Grafana

Need help? [Documentation](#) [Tutorials](#) [Community](#) [Public Slack](#)

Basic

The steps below will guide you to quickly finish setting up your Grafana installation.

TUTORIAL

DATA SOURCE AND DASHBOARDS

Grafana fundamentals

Set up and understand Grafana if you have no prior experience. This tutorial guides you through the entire process and covers the "Data source" and "Dashboards" steps to the right.

Learn how in the docs

COMPLETE

Add your first data source

Learn how in the docs

DASHBOARDS

Create your first dashboard

Learn how in the docs

Dashboards

Starred dashboards

Recently viewed dashboards

Latest from the blog

Grafana

Circumnavigation or bust:

Now in the Grafana Dashboard select **Add your first data source.**

Basic

The steps below will guide you to quickly finish setting up your Grafana installation.

TUTORIAL

DATA SOURCE AND DASHBOARDS

Grafana fundamentals

Set up and understand Grafana if you have no prior experience. This tutorial guides you through the entire process and covers the "Data source" and "Dashboards" steps to the right.

Learn how in the docs

COMPLETE

Add your first data source

Learn how in the docs

Search for InfluxDB, select it

Home > Connections > Data sources > Add data source

Add data source

Choose a data source type



InfluxDB
Open source time series database
Core

Change the Query language from **InfluxQL to Flux** and enter the details as **URL - <http://influxdb:8086> , Organization - my-org , Token - mysecrettoken123 , Bucket - iotdb** . After entering the details click Save and Test .

Query language



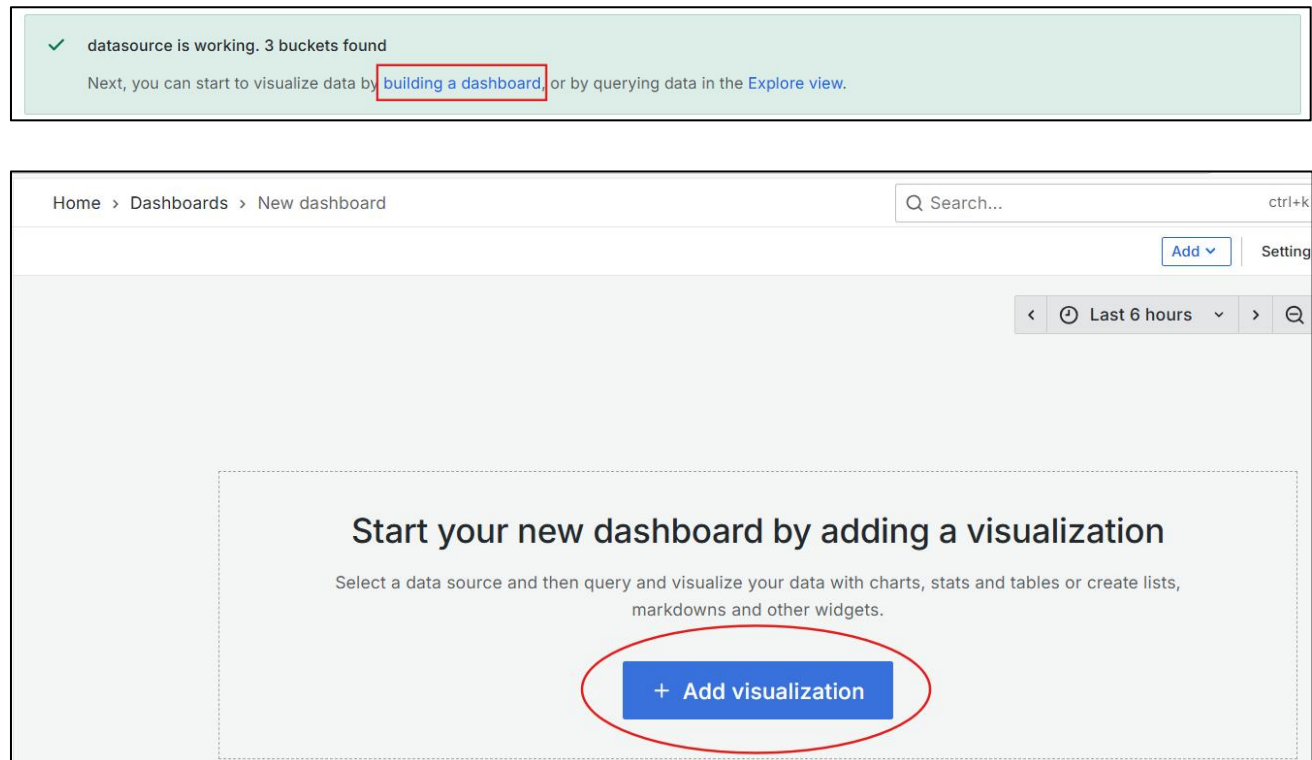
InfluxQL
The InfluxDB SQL-like query language.

SQL
Native SQL language. Supported in InfluxDB 3.0

Flux
Supported in InfluxDB 2.x and 1.8+

HTTP

Now you can see a message saying “datasource is working. 3 buckets found” now select buliding a dahsboard and redirected page will appear.Select Add visualization .



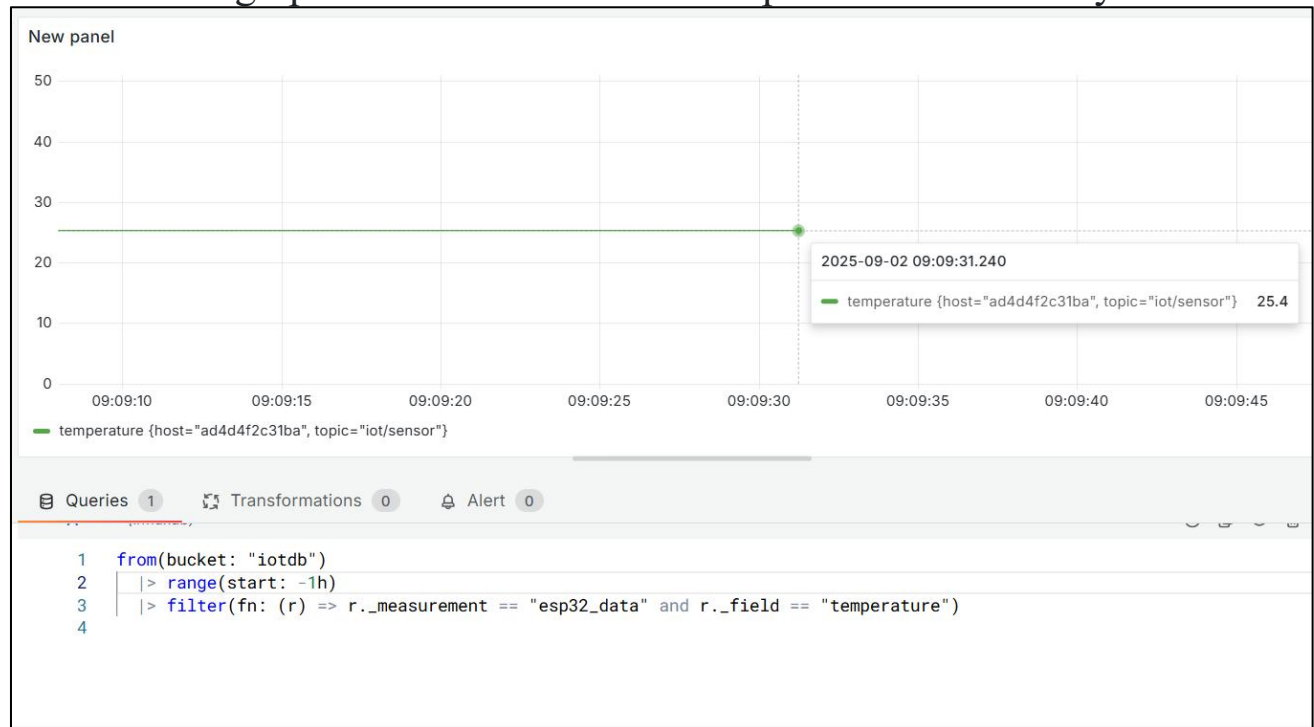
Select InfluxDB as Data Source



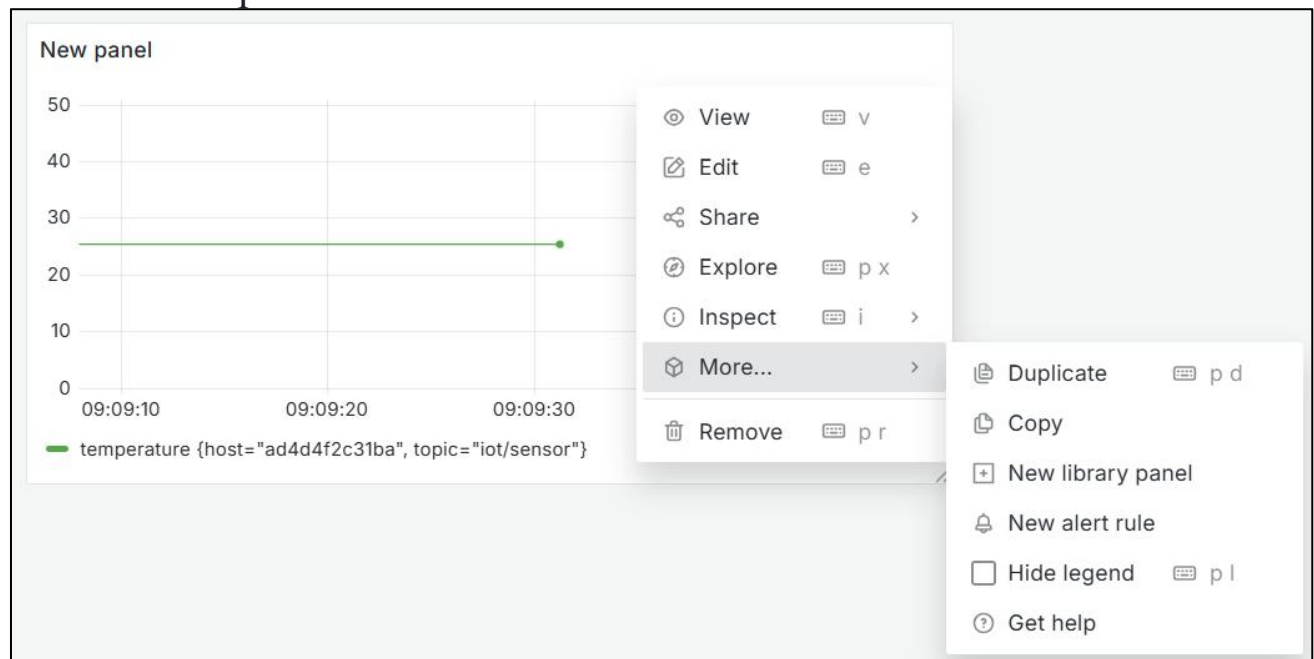
Now at the bottom enter the Flux query snippet to fetch temperature and click refresh.Enter the command provided

```
from(bucket: "iotdb")
  |> range(start: -1h)
  |> filter(fn: (r) => r._measurement == "esp32_data" and r._field == "temperature")
```

Now we can observe the temperature we sent thorough Command Prompt is visualised in graph. We have to do the same process for humidity too.



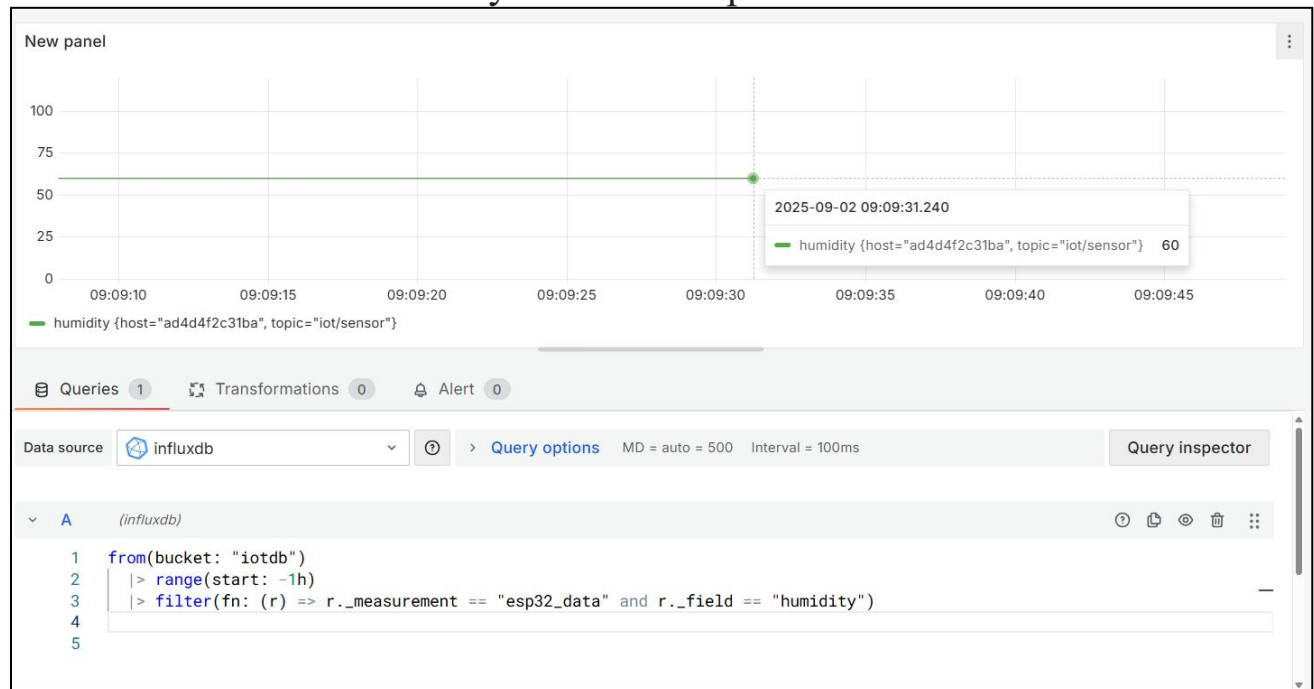
Now come back to the created dashboard Duplicate the panel and enter to the duplicated panel by choosing edit option. You can see triple dots at the corner of the panel.



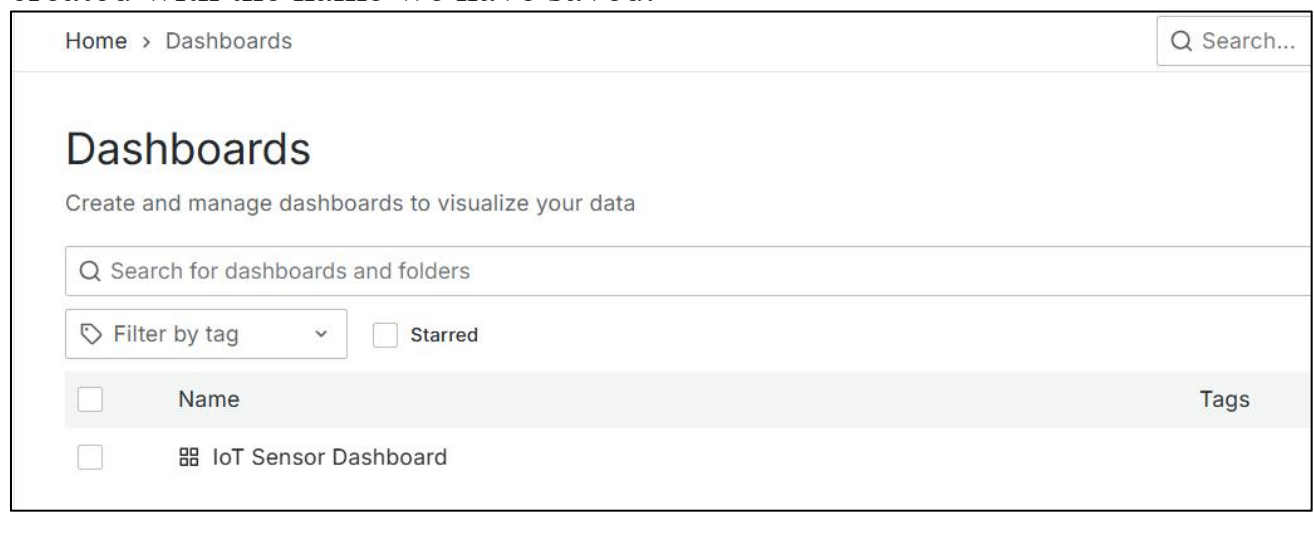
Now enter the Flux query snippet to fetch humidity and click refresh

```
from(bucket: "iotdb")
  |> range(start: -1h)
  |> filter(fn: (r) => r._measurement == "esp32_data" and r._field == "humidity")
```

We can observe the humidity recorded as per time



Edit their panel titles such as Temperature Over Time and Humidity Levels for both the graphs and select Save Dashboard , enter title as IoT Sensor Dashboard and click Save. Now in the dashboards Section we can see the created with the name we have saved.



If it is necessary to clean the container then use the command in the Command Prompt - **[docker-compose down](#)** This command stops and removes the containers created by Docker Compose (InfluxDB, Mosquitto, Telegraf, Grafana). The data stored in Docker volumes remains intact, so your InfluxDB data and Grafana settings are preserved. If you want to completely remove all data including stored sensor data and configurations (this step is irreversible), run - **[docker-compose down -v](#)** The -v flag also removes all named volumes defined in your Docker Compose file. This deletes your InfluxDB data, Grafana data, and any persisted MQTT broker data. Use this if you want a clean start or no longer need the data.

All Done - You now have a working IoT data pipeline using MQTT, Telegraf, InfluxDB, and Grafana.

Docker Installation Windows - [Video](#)

Docker Compose Installation - [Video](#)

Sending Sensor Values using TIG Stack

Open Command Prompt and enter to the location where the repository is cloned. Now enter the command to start Docker Engine - `docker-compose up -d`

```
Command Prompt
Microsoft Windows [Version 10.0.26100.5074]
(c) Microsoft Corporation. All rights reserved.

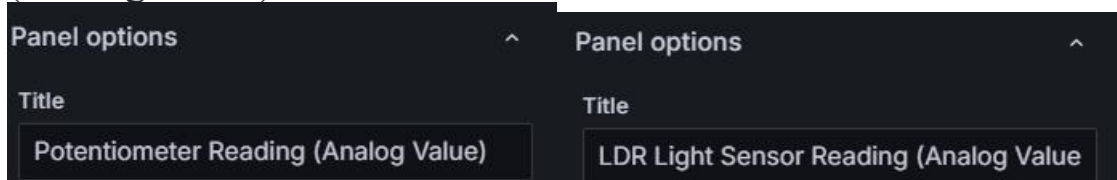
C:\Users\super>cd iot_tig_stack_with_mqtt_using_docker

C:\Users\super\iot_tig_stack_with_mqtt_using_docker>docker-compose up -d
time="2025-09-03T01:26:52+05:30" level=warning msg="C:\\Users\\super\\iot_tig_stack_with_mqtt_using_docker\\docker-compo
se.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Running 6/6
✔ Network iot_tig_stack_with_mqtt_using_docker_default      Created      0.1s
✔ Volume "iot_tig_stack_with_mqtt_using_docker_grafana_data" Created      0.0s
✔ Volume "iot_tig_stack_with_mqtt_using_docker_influxdb2_data" Created      0.0s
✔ Container grafana                                          Started      1.4s
✔ Container influxdb                                         Started      1.3s
✔ Container telegraf                                         Started      1.4s

C:\Users\super\iot_tig_stack_with_mqtt_using_docker>
```

Now the Docker Engine is started. Click on <http://localhost:3000/> to enter your Grafana Dashboard. Follow the steps to add visualization which is explained in the TIG Stack Installation explanation.

We need two dashboards so duplicate them and enter the title as “Potentiometer Reading (Analog Value)” & “LDR Light Sensor Reading (Analog Value)”.



Go to your MQTT Explorer and connect to HiveMQ public client (You only need to subscribe to one topic ("iot/sensor") because both the potentiometer and LDR values are sent together as a single JSON payload in that one topic message. This simplifies the pipeline by aggregating sensor data into one message per interval. If you sent these sensors to separate topics, you would need multiple subscriptions and then merge the data on the database side, complicating the setup. In this design, Telegraf's MQTT consumer plugin receives one JSON message containing both "pot" and "ldr" fields from the single topic, parses it, and writes it to InfluxDB in one consistent data point each interval.). Go to Arduino IDE and start writing the program. I have given the code for clear understanding.

sketch_sep2a.ino

```
#include <WiFi.h>
#include <PubSubClient.h>

// WiFi credentials
const char* ssid = "S23FE";
const char* password = "Sanjey123@";

// MQTT broker info (HiveMQ public broker)
const char* mqtt_server = "broker.hivemq.com";
const int mqtt_port = 1883;
const char* mqtt_topic = "iot/sensor";

// Analog pins for sensors
const int potPin = 36; // Example analog pin for potentiometer
const int ldrPin = 39; // Example analog pin for LDR

WiFiClient espClient;
PubSubClient client(espClient);

void setup_wifi() {
  delay(10);
  Serial.println();
  Serial.print("Connecting to WiFi: ");
  Serial.println(ssid);

  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("");
  Serial.println("WiFi connected");
  Serial.print("IP address: ");
  Serial.println(WiFi.localIP());
```

```

}

void reconnect() {
  // Loop until reconnected
  while (!client.connected()) {
    Serial.print("Connecting to MQTT...");
    if (client.connect("ESP32Client")) {
      Serial.println("connected");
    } else {
      Serial.print("failed, rc=");
      Serial.print(client.state());
      Serial.println(" try again in 5 seconds");
      delay(5000);
    }
  }
}

void setup() {
  Serial.begin(115200);
  setup_wifi();
  client.setServer(mqtt_server, mqtt_port);
}

void loop() {
  if (!client.connected()) {
    reconnect();
  }
  client.loop();

  // Read analog values
  int potValue = analogRead(potPin);
  int ldrValue = analogRead(ldrPin);

  // Construct JSON payload
  String payload = "{";
  payload += "\"pot\":";
  payload += potValue;

```

```

payload += ",";
payload += "\"ldr\":";
payload += ldrValue;
payload += "}";

Serial.print("Publishing message: ");
Serial.println(payload);

client.publish(mqtt_topic, payload.c_str());

delay(10000); // Wait 10 seconds (should match Telegraf interval)
}

```

Upload the code and open the Serial Monitor of your Arduino IDE, you can observe the Values of both POT and LDR displayed.

```

02:06:58.302 -> Publishing message: {"pot":692,"ldr":763}
02:07:08.270 -> Connecting to MQTT...connected
02:07:08.953 -> Publishing message: {"pot":711,"ldr":756}
02:07:18.959 -> Connecting to MQTT...connected
02:07:20.021 -> Connecting to MQTT...connected
02:07:20.793 -> Publishing message: {"pot":674,"ldr":783}
02:07:30.823 -> Connecting to MQTT...connected
02:07:31.446 -> Publishing message: {"pot":721,"ldr":751}

```

Go to your Grafana Dashboard and for both the dashboards created enter the Flux command

for Potentiometer

from(bucket: "iotdb")

```

|> range(start: -1h)
|> filter(fn: (r) => r._measurement == "esp32_data")
|> filter(fn: (r) => r._field == "pot")
|> aggregateWindow(every: 10s, fn: mean, createEmpty: false)
|> yield(name: "mean_pot")

```

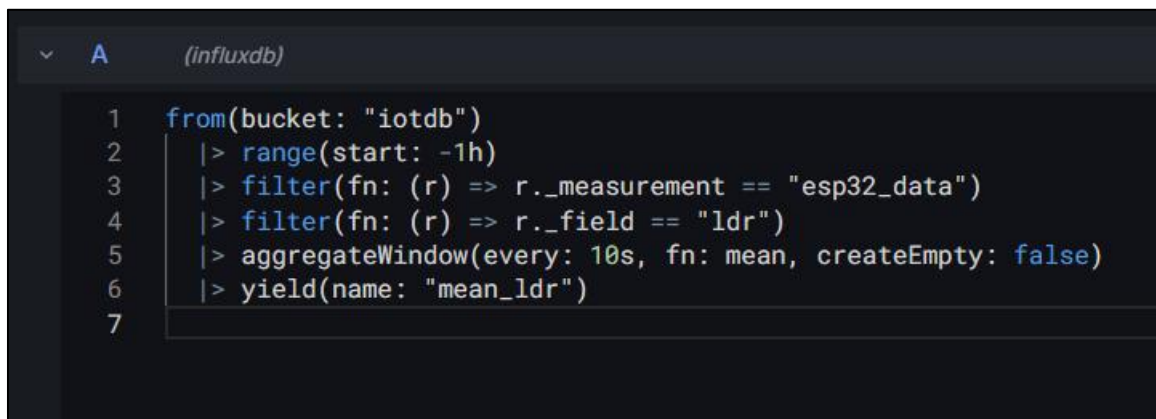
```

A (influxdb)
1 from(bucket: "iotdb")
2   |> range(start: -1h)
3   |> filter(fn: (r) => r._measurement == "esp32_data")
4   |> filter(fn: (r) => r._field == "pot")
5   |> aggregateWindow(every: 10s, fn: mean, createEmpty: false)
6   |> yield(name: "mean_pot")
7

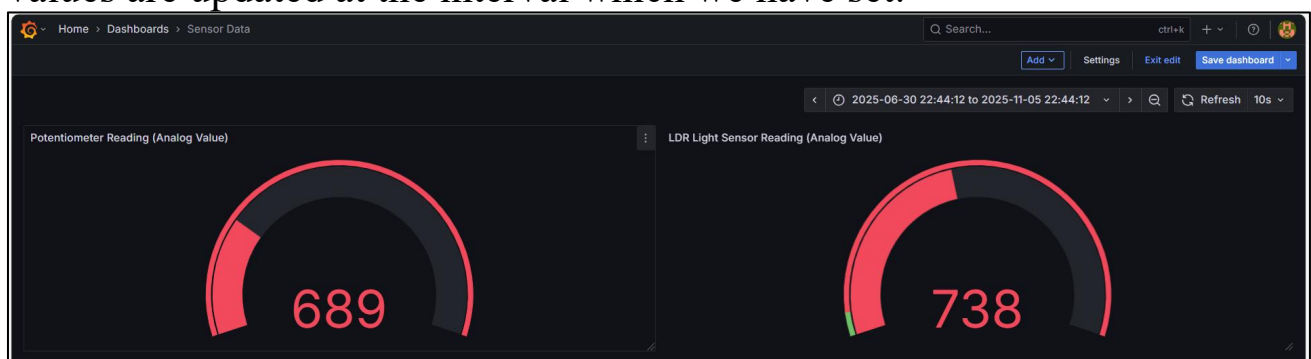
```

for LDR

```
from(bucket: "iotdb")
  |> range(start: -1h)
  |> filter(fn: (r) => r._measurement == "esp32_data")
  |> filter(fn: (r) => r._field == "ldr")
  |> aggregateWindow(every: 10s, fn: mean, createEmpty: false)
  |> yield(name: "mean_ldr")
```

A screenshot of a terminal window with a dark background. The title bar shows a dropdown arrow, the letter 'A', and '(influxdb)'. The terminal contains a Flux query with line numbers 1 through 7 on the left margin. The query is: 1 from(bucket: "iotdb") 2 |> range(start: -1h) 3 |> filter(fn: (r) => r._measurement == "esp32_data") 4 |> filter(fn: (r) => r._field == "ldr") 5 |> aggregateWindow(every: 10s, fn: mean, createEmpty: false) 6 |> yield(name: "mean_ldr") 7

After entering the Flux commands and click refresh button(set auto refresh interval such as 5seconds, 10 seconds etc., so different values are displayed) you can see the same values which are seen in the serial monitor are displayed in our Grafana Dashboard too. Save the Dashboard as “Sensor data”.Now the complete dashboard is displayed and hence you can see the values are updated at the interval which we have set.



All Done - You now have a working model to display the sensor data in the Grafana dashboard !!